

Piyushwani — Scope of Improvement

A review of the Next.js 16 / React 19 port of the Piyushwani (OPC) Private Limited website, identifying what stands between the current build and a production-ready pharmaceutical marketing platform.

PROJECT	STACK	CODEBASE	ROUTES	DATE
wani-main	Next.js 16.3 · React 19.2 · TypeScript 5	~4,960 lines TS/TSX · 956 lines CSS	26 prerendered	11 August 2026

3 BLOCKERS	5 HIGH	9 MEDIUM	7 LOW	3 REGULATORY
---------------	-----------	-------------	----------	-----------------

Baseline is sound. The build is clean — TypeScript strict mode passes with no errors, ESLint is clean, all 26 routes prerender, and every route returns the correct status code. The design system ported faithfully, images are optimised through `next/image`, and the API layer already fails honestly rather than faking success. The findings below are about what has *not been built yet*, not about repairing what has.

01 Blockers — the site cannot go live without these

Each of these prevents the site from performing its core commercial function. They are sequenced first because everything else is cosmetic until they are resolved.

B1

There is no backend — every dynamic surface is inert

BLOCKER

Batch verification, the quote request form, the P-Wanicare signup and admin sign-in all call endpoints under `/api/v1/*`. No such handlers exist in this repository — there is no `app/api/` directory and no separate service. In its current state every verification attempt returns “we couldn’t reach the verification service”, both forms fail to submit, and no one can log into the console.

This is by design — the port deliberately kept the front end free of batch data and passwords — but it means the site is presently a brochure that cannot verify anything.

WHAT THIS NEEDS

A datastore for batches, products, CMS fields and enquiries, plus handlers for the six endpoint groups declared in `lib/config.ts`. These can be Next Route Handlers inside `app/api/` (keeping one deployable) or a separate service pointed at by `NEXT_PUBLIC_API_BASE`. The response shape for verification is already typed as `Batch` in `lib/batches.ts`, so the contract is settled.

B2 QR codes printed on packs would land on a 404

BLOCKER

The admin console prints `Verification URL /verify/{code}` on every QR label panel. That route does not exist — the app has only `app/verify/page.tsx`, the manual entry screen. There is no `app/verify/[code]/page.tsx`, so scanning a pack would open a “We can’t find that page” error.

This is the single most urgent item in the document. Batch verification is the company’s stated differentiator, and **packaging is printed once**. If labels are ordered against a URL pattern the site does not serve, the error is physically embedded in stock.

WHAT THIS NEEDS

A dynamic route that reads the code from the URL, fetches the record server-side (so the page is shareable, indexable and works without JavaScript), and renders the existing `BatchRecord` component. Keep the typed-entry terminal as the fallback for damaged codes. Settle this before any label artwork is approved.

B3 The admin console is publicly reachable and gated only in the browser

BLOCKER

`/admin` is statically prerendered and served to any visitor. The gate is React state — setting `authenticated` to true is all that stands between a visitor and the console shell. There is no `middleware.ts`, no server-side route protection and no role checks.

Today the exposure is contained because the console holds no live data. That changes the moment it reads real content, batches or enquiries. Separately, the session token is kept in `sessionStorage`, which any injected script can read.

WHAT THIS NEEDS

Server-side auth enforcement — middleware or a server component check that redirects unauthenticated requests before any console markup is sent. Move the token to an httpOnly, SameSite cookie. Add role/permission checks matching the `User / Role` model the console’s own copy already references.

02 High — data loss, broken links and user-facing defects

H1 The quote form silently discards two fields the user filled in

HIGH

`RfqForm.tsx` renders inputs for *Company / clinic / pharmacy* and *Approximate quantity*, stores both in state, and then omits both from the POST body — only name, email, phone, type, message and consent are sent.

A distributor who types their company name and an order volume of 50,000 units loses both before the enquiry reaches anyone. Sales then has to email back and ask for information the customer already gave. (This behaviour was inherited from the original prototype and carried through the port unchanged.)

WHAT THIS NEEDS

Add `organisation` and `quantity` to the request payload and to the inbox columns in the console — or remove the two inputs if they are genuinely not wanted.

H2 Product URLs are array positions, so links repoint when the catalogue changes

HIGH

Product pages are addressed as `/product/5`, derived from the item's index in the `PRODUCTS` array. Inserting, reordering or deleting a single product silently reassigns every URL after it.

For a medicine catalogue this is more than an SEO problem: a bookmarked link, a printed reference or an indexed search result that pointed at Paracetamol can start resolving to Cetirizine, with no error and no redirect.

`lib/products.ts` already computes a `slug` in `toProduct()`, but nothing consumes it.

WHAT THIS NEEDS

Move to stable slugs (`/product/paracetamol-500mg-tablets-ip`) sourced from the product record rather than its position, and keep the slug immutable once published.

H3 Refreshing the admin console logs the operator out mid-task

HIGH

`authed` initialises to `false` and the existing token in `sessionStorage` is never read on mount. Any page reload — or an accidental `F5` — drops the operator back to the sign-in screen and discards whatever they had open.

WHAT THIS NEEDS

Restore the session from the stored credential on load (and validate it against the server), so a refresh resumes rather than restarts.

H4 No error or loading boundaries anywhere in the app

HIGH

There is no `error.tsx`, `global-error.tsx` or `loading.tsx` in any route segment. A render-time exception falls through to Next's default error screen — unbranded, uninformative, and a poor look for a regulated business. Route transitions that wait on data have no skeleton state.

WHAT THIS NEEDS

A branded error boundary with a recovery action, a root `global-error`, and loading states on any route that will fetch once the API lands (verification above all).

H5 Form validation errors are invisible to screen readers

HIGH

Validation messages are revealed purely by a CSS class — `.f.bad .err{display:block}`. The fields carry no `aria-invalid`, no `aria-describedby` linking the message to the input, and nothing announces the change. A screen-reader user who submits the quote form with a bad phone number is told only that nothing happened.

Focus is also not moved to the first invalid field, so keyboard users must hunt for the problem.

WHAT THIS NEEDS

Wire `aria-invalid` and `aria-describedby` on each field, mark the error container as a live region, and move focus to the first failure on submit.

03 Medium — discoverability, accessibility and hardening

M1 No SEO infrastructure

MEDIUM

There is no `sitemap.ts`, no `robots.ts`, no `metadataBase`, no canonical URLs, and no Open Graph or Twitter card metadata. Every page has a good title and description, but sharing one on WhatsApp or LinkedIn — the channels a distributor actually uses — produces a bare, image-less link. `/admin` is correctly marked `noindex`; nothing else declares anything.

WHAT THIS NEEDS

Sitemap and robots routes, `metadataBase` plus canonicals in the root layout, and an OG image per major page.

M2 No structured data for a company selling verifiable provenance

MEDIUM

No JSON-LD anywhere. For this business specifically it is high leverage: `Organization` carries the registered identity and licences, `Product / Drug` carries composition, dosage form and manufacturer. This is the mechanism by which search engines surface exactly the provenance data the company is differentiating on.

WHAT THIS NEEDS

`Organization` on the root layout; `Product` or `Drug` on product pages; `BreadcrumbList` alongside the existing visual breadcrumbs.

M3 No favicon, app icons or web manifest

MEDIUM

There is no `app/icon`, `apple-icon` or `manifest.ts`. Browser tabs and mobile bookmarks fall back to a default globe. An unused `public/images/logo.jpg` was extracted from the prototype and is never referenced.

WHAT THIS NEEDS

Icon set generated from the brand mark (already an inline SVG in `components/icons.tsx`) plus a manifest for installability.

M4 Images ship without size hints or placeholders

MEDIUM

No `next/image` in the project sets a `sizes` attribute. Because the CSS scales every image to its container, the browser cannot tell which srcset candidate it needs and can download a substantially larger file than it will display — most costly on the hero and the four category tiles. Nothing uses `placeholder="blur"`, so images pop in against grey boxes. Source assets total 1.7 MB.

WHAT THIS NEEDS

Accurate `sizes` per layout slot, blur placeholders on the large above-fold images, and AVIF/WebP output enabled in `next.config.ts`.

M5 The team tabs are only half-wired for assistive technology

MEDIUM

`TeamTabs` declares `role="tablist"`, `role="tab"` and `aria-selected`, but the three panels have no `role="tabpanel"` and no `id` / `aria-controls` relationship, so a screen reader cannot associate a tab with what it reveals. Arrow-key navigation — the expected keyboard model for a tablist — is not implemented.

WHAT THIS NEEDS

Complete the ARIA relationship and add arrow-key/Home/End handling, or drop the tab roles and present the three groups as plain headed sections.

M6 Several prominent calls to action lead nowhere

MEDIUM

"Shop on WaaniGo" and "Buy on WaaniGo" both link to `/contact`. "Download certificate" on every certification card and "Download COA" on batch records do the same. They are styled as functional buttons and behave as dead ends — the commercial path off this site is not actually wired.

WHAT THIS NEEDS

Point the WaaniGo buttons at the real store once it exists; serve actual certificate and COA PDFs, or relabel the buttons to describe what they really do ("Request a copy").

M7 Public forms and the verification endpoint have no abuse protection

MEDIUM

Neither public form carries a honeypot, captcha or client-side throttle, and no rate limiting is specified for verification. Verification is the endpoint an attacker would hammer: although codes come from a deliberately unguessable 30-symbol alphabet, an unthrottled lookup endpoint still invites enumeration, and the console's own copy identifies clustered failed scans as the earliest counterfeit signal.

WHAT THIS NEEDS

Server-side rate limiting on all four public endpoints, a honeypot field on both forms, and logging of failed verification attempts with enough context to spot a cluster.

M8 Content is compiled in, so the console cannot actually publish

MEDIUM

`Ph` and `cmsLookup` read a module-level constant in `lib/cms.ts`. Filling one of the amber `[CLIENT: ...]` gaps therefore requires a code edit and a redeploy. Console edits are held in session state and vanish on reload — the gap between the demo and a working CMS.

WHAT THIS NEEDS

Back the CMS with the content API and read it in server components, with revalidation on save so edits reach the storefront without a rebuild.

M9 No security headers configured

MEDIUM

`next.config.ts` is empty. No Content-Security-Policy, HSTS, `X-Frame-Options`, `Referrer-Policy` or `X-Content-Type-Options` is set. For a site that will handle enquiry data and an authenticated console, these are baseline.

WHAT THIS NEEDS

A headers block in the Next config, with CSP tightened once the API origin is known.

ID	FINDING	DETAIL AND REMEDY
L1	No tests, no CI	No test runner, no component or end-to-end coverage, no pipeline. The verification state machine — verified, expiring, expired, withdrawn, not-found, transport-failure — is precisely the logic that warrants a suite, since a wrong branch tells someone an expired medicine is fine. Add Vitest + Testing Library for units, Playwright for the verify and quote journeys, and run both on every push.
L2	Not under version control	The project directory is not a git repository, so there is no history, no review trail and no rollback. Initialise, commit the current state as a baseline, and adopt branch-per-change.
L3	No analytics or error tracking	Nothing records page views, conversion on the quote form, or client-side exceptions. Notably, failed verification scans — described in the console as the earliest signal of counterfeit stock — are captured nowhere. Add product analytics plus an error reporter such as Sentry.
L4	Unused asset	<code>public/images/logo.jpg</code> (42 KB) was extracted from the prototype and is never referenced; the brand mark renders as inline SVG. Remove it or adopt it for the icon set in M3.
L5	Single locale	Fixed <code>lang="en-IN"</code> with no i18n scaffolding. For a consumer-facing Indian pharmaceutical brand, Hindi and regional languages are a realistic requirement — cheaper to structure for now than to retrofit.
L6	Copyright year is build-stamped	The footer year is fixed at generation time, so it only refreshes on redeploy. Harmless if the site ships regularly; worth a dynamic segment if it will sit untouched.
L7	No environment validation	<code>NEXT_PUBLIC_API_BASE</code> falls back silently to an empty string, so a typo or a missing value degrades quietly to same-origin instead of failing loudly at boot. Validate env at startup with a schema.

05 Regulatory and content readiness

These are not engineering defects, but they gate launch for a pharmaceutical business and are visible in the codebase.

R1	All three legal pages are unreviewed drafts	HIGH
Privacy, Terms and Disclaimer each render a standing banner reading “Draft for legal review ... not legal advice”. The privacy text asserts compliance with the Digital Personal Data Protection Act 2023 and names a Grievance Officer with a 48-hour acknowledgement commitment. Publishing these undertakings without counsel sign-off creates real exposure. WHAT THIS NEEDS Legal review and sign-off, then removal of the draft banner.		
R2	No adverse-event or product-complaint reporting path	MEDIUM
The site offers a general quote-request form and a support email. There is no dedicated route for reporting an adverse reaction or a product complaint, and no pharmacovigilance contact — a normal expectation for a pharmaceutical marketer, and the natural companion to a verification tool that already tells users to stop using a product and get in touch. WHAT THIS NEEDS A distinct reporting form with its own routing, retention rules and acknowledgement, linked from the verify and product pages.		

R3 Thirteen of twenty-four content fields are still unfilled

MEDIUM

The CMS carries 24 defined fields; 11 hold values. The 13 outstanding include the drug licence number, contract manufacturer name, manufacturing site address and manufacturing licence number — every one of which is printed on a batch verification record, plus the founder biography and the entire P-Wanicare brand description.

The site handles this well: gaps render as amber `[CLIENT: ...]` chips rather than shipping silently blank, and the console badges the count. But a verification record that names no manufacturer does not do the job it exists to do.

WHAT THIS NEEDS

Client content collection, prioritising the four compliance fields that appear on batch records.

06 Suggested sequence

Ordered by dependency rather than effort — later phases assume earlier ones. The verification deep link (B2) is called out first because packaging decisions depend on it and cannot be reversed once printed.

PHASE	OBJECTIVE	ITEMS	WHY HERE
1	Settle the URL contract	B2, H2	Decide the verification and product URL shapes before labels or printed material are commissioned. These are the only decisions in this document that become expensive to reverse.
2	Build the backend	B1, B3, M8	Everything dynamic depends on it. Ship auth server-side from the outset rather than retrofitting protection onto a console that is already public.
3	Correctness and resilience	H1, H3, H4, M7, M9	Stop losing enquiry data, stop logging operators out, and put boundaries and rate limits in place before real traffic arrives.
4	Accessibility and reach	H5, M1, M2, M3, M4, M5	Once the site works, make it findable, shareable and usable by everyone. Structured data is unusually valuable here given the provenance positioning.
5	Launch readiness	R1, R2, R3, M6	Legal sign-off, the reporting path, and the outstanding client content — the remaining gates between a working site and a publishable one.
6	Engineering practice	L1–L7	Version control and tests ideally start at phase 1; they are listed last only because nothing above is blocked on them.

One decision worth taking early. B2 and H2 both concern URL shape, and both become materially harder to change after launch — B2 because codes are printed onto physical packaging, H2 because product links get bookmarked, shared and indexed. Neither is difficult to implement now. Both are expensive to correct later.

07 What is already in good shape

Recorded so that remediation does not disturb it.

- **Honest failure states.** `apiFetch` never throws and never fabricates success; verification distinguishes “no record of this code” from “we couldn’t reach the service”, and both differ from a verified result. For a counterfeiting control, this distinction is the entire point.
- **No secrets or batch data in the client.** The port deliberately holds neither, so the front end has nothing worth extracting.
- **Content gaps are visible rather than silent.** Unfilled fields render as amber chips and are counted in the console — nothing ships blank by accident.
- **Type safety throughout.** TypeScript strict mode passes with no errors and no `any` escapes in the application code.
- **Correct rendering strategy.** All 26 routes prerender as static content; interactivity is isolated to the few components that genuinely need it.
- **Motion and contrast were considered.** `prefers-reduced-motion` is honoured throughout, and the palette was designed against AA contrast targets.
- **The admin console resists a real trap.** Its modules are declared at module scope, so open editor drafts survive re-renders instead of being silently discarded.